

12 | Kubernetes HPA：云原生应用扩缩容管理之道

潘野 · 云原生基础架构实战课



你好，我是潘野。

上一讲的末尾我讲了公有云上引起浪费的两个大类情况，其实云资源浪费是一个普遍存在的问题，无论是在公有云还是私有云环境中都可能发生，如何解决云资源浪费也是基础架构管理中非常重要的一个部分。

今天我们展开讲讲，哪些原因会造成云资源浪费，以及我们怎么利用好云原生技术来解决云资源的浪费问题，提升我们的资源使用率。

哪些情况会造成资源浪费？

云资源利用率低是指已分配的云资源没有被充分利用，导致闲置浪费。我们看看哪些情况会造成这个结果。

首先是过度或错误配置。在创建云实例时，为了满足峰值需求，应用维护者往往会配置过多的资源。然而，实际应用中，资源需求通常会低于峰值，导致大部分时间资源处于闲置状态。

比如我就遇到过这种情况：开发申请了十台 300G 内存的机器做 Redis Cluster，而上线之后，发现实际使用率只有 20%，这就相当于 8 台机器被浪费了。

错误配置的现象也很常见，配置云资源时，由于缺乏经验或相关知识，可能会出现错误配置。例如应用程序比较消耗 CPU，不太消耗内存，但是却申请了一个内存很大的机器，导致资源浪费。

然后是资源预留问题。为了保证资源的可用性，应用维护者会预留一定量的资源保障应用做横向或者纵向扩展。然而，预留资源往往无法完全利用，导致资源闲置浪费。

最后是不必要的服务，有时候我们可能会启用不必要的云服务，导致资源浪费。

解决办法

在云原生体系中，解决上面这些问题并不难做，我们解决的思路是让应用动态地调整资源使用。在流量高峰时期，又能及时动态扩容上去。当流量进入低峰时候，又能释放掉闲置资源。用一个词来总结就是**动态扩缩容**。

动态扩缩容里主要包括两个层级的动态扩缩容。

第一个层级是应用本身**水平扩展与纵向扩展**。

水平扩展是指通过增加节点数量来扩展应用的容量。例如，可以将一个应用部署在多个服务器上，每个服务器都运行应用的一部分。水平扩展可以有效地提高应用的处理能力，并可以应对更大的负载。

纵向扩展是指通过增加单个节点的资源来扩展应用的性能。例如，可以增加服务器的 CPU、内存或存储空间。纵向扩展可以提高应用的处理速度，并可以处理更复杂的任务。

在应用本身这个层级的动态扩缩容，我们可以使用 Kubernetes HPA 和 VPA 来实现应用层面的动态扩缩容。

第二个层级是 Kubernetes 集群本身容量的动态扩缩容，指根据集群的负载情况自动增加或减少节点数量，以满足应用的需求。这样做可以有效地提高资源利用率，并降低成本。

在这个层级的动态扩缩容中，可以使用 Kubernetes 官方社区支持的集群自动扩缩容工具 Cluster Autoscaler，它可以根据 Pod 的资源需求和节点的资源可用情况自动扩缩容集群。

今天我们主要讲解第一个层级，应用本身的**水平扩展与纵向扩展**。

HPA

我们先来看 HPA，也就是水平扩缩容。HPA 可以根据 CPU 利用率、内存利用率和其他自定义指标自动扩缩容 Pod 数量，以满足应用程序的负载需求。

HPA 工作原理是这样的，控制器会定期（默认每 30 秒）查询目标资源的 Pod 的资源使用情况，并将其与 HPA 对象中指定的指标进行比较。如果资源使用情况超过或低于目标指标，HPA 控制器会根据扩缩容策略来扩缩容 Pod 数量。

Kubernetes 默认支持根据容器的 CPU 和内存的使用率。我们结合例子来理解一下。

 复制代码

```
1 apiVersion: autoscaling/v2beta2
2 kind: HorizontalPodAutoscaler
3 metadata:
4   name: web-app-hpa
5 spec:
6   scaleTargetRef:
7     apiVersion: apps/v1
8     kind: Deployment
9     name: web-app
10  minReplicas: 1
11  maxReplicas: 10
12  metrics:
13  - type: Resource
14    resource:
15      name: cpu
```

从上述配置可以看到，我们针对 web-app 这个 deployment 设置了最小副本数 1，最大副本数 10，在 CPU 平均使用率 80% 的时候开始扩容。

基于 CPU 和内存指标这种配置非常简单，但是这在实际生产中并不够用。我们举个常见例子，比如在一个生产者消费者服务里，当消费者的消费速度不足，我们希望启动更多的消费者服务的时候，CPU 和内存的使用率这种指标显然就不能满足需求了。

怎么扩展 HPA 的能力

现在的 HPA 支持了从其他的 API 中获取指标来进行扩容，Kubernetes API 中有一个资源叫 `apiservices`，用于注册和管理非核心 API，非核心 API 是指 Kubernetes 核心代码之外的 API。这些 API 可以由第三方开发人员提供，也可以由 Kubernetes 社区提供。

HPA 控制器会从 `apiservices` 管理的 API 中获取一些指标，然后根据定义好指标的阈值来触发一些扩缩容。我们可以用 `kubectl get apiservices` 这个命令，查看集群现在支持的第三方 API。

 复制代码

```
1 >>> kubectl get apiservices.apiregistration.k8s.io
2 v1beta1.custom.metrics.k8s.io           monitoring/prometheus-adapter
3 v1beta1.external.metrics.k8s.io         addons-system/keda-operator-metrics-apiserver
4 v1beta1.metrics.k8s.io                  kube-system/metrics-server
```

对于资源指标，使用 `metrics.k8s.io` [API](#)，一般由 [metrics-server](#) 提供。它可以作为集群插件启动。

对于自定义指标，使用 `custom.metrics.k8s.io` [API](#)。它由其他“适配器 (Adapter)” API 服务器提供。从上面的命令输出可以看出，这里是由 `prometheus-adapter` 来提供的。

对于外部指标，将使用 `external.metrics.k8s.io` [API](#)。

接下来，我们来看看怎么使用 Prometheus-Adapter 来扩展 HPA 的能力。

Prometheus-Adapter

Prometheus-Adapter 是一款用于将 Prometheus 指标转换为 Kubernetes 自定义指标的工具。它可以用于将 Prometheus 监控的应用程序的指标暴露给 Kubernetes Horizontal Pod Autoscaler (HPA) 等工具。

Prometheus-Adapter 的工作过程要经历三个阶段。

1. Prometheus-Adapter 会定期（默认每 30 秒）从 Prometheus 服务器拉取指标数据。
2. Prometheus-Adapter 会将拉取到的指标数据转换为 Kubernetes API Server 可以理解的形式。
3. Prometheus-Adapter 会将转换后的指标数据暴露给 Kubernetes API Server。

假设我们有一个 Web 应用程序，我们希望通过 Prometheus 监控这个应用的性能指标。在这个前提下，我们就可以使用 HPA 配合 Prometheus-Adapter 来自动扩缩容 Pod 以满足应用程序的负载需求。

接下来，我们再通过一个例子，了解一下 Prometheus-Adapter 是怎么实现 HPA 扩展的。

首先，我们需要安装 Prometheus 和 Prometheus-Adapter，这里你参照 [🔗 官方文档](#) 安装即可。

然后，我们需要在 Prometheus-Adapter 的 configmap 中加上我们需要监控的指标，这里是从 Prometheus 查询 HTTP 请求总数，排除某些指标，用 "_qps" 后缀重命名，并计算 30 秒窗口内的查询率。

 复制代码

```
1  custom:
2    - seriesQuery: '{__name__=~"^http_requests.*_total$",container!="POD",namespac
3      resources:
4        overrides:
5          namespace: { resource: "namespace" }
```

```
6         pod: { resource: "pod" }
7     name:
8         matches: "(.*)_total"
9         as: "${1}_qps"
10    metricsQuery: sum(rate(<<.Series>>{<<.LabelMatchers>>}[30s])) by (<<.GroupBy
```

一切都成功的话，我们运行下面这条命令即可获得对应的输出。

 复制代码

```
1 kubectl get --raw '/apis/custom.metrics.k8s.io/v1beta1/namespaces/default/pods/*/
```

最后就是 HPA 的配置了，你可以参考后面的例子。

 复制代码

```
1 kind: HorizontalPodAutoscaler
2 apiVersion: autoscaling/v2
3 metadata:
4   name: sample-httpserver
5 spec:
6   scaleTargetRef:
7     apiVersion: apps/v1
8     kind: Deployment
9     name: sample-httpserver
10  minReplicas: 1
11  maxReplicas: 10
12  behavior:
13    scaleDown:
14      stabilizationWindowSeconds: 30
15      policies:
16        - type: Percent
17          value: 100
18          periodSeconds: 15
19    scaleUp:
20      stabilizationWindowSeconds: 0
21      policies:
22        - type: Percent
23          value: 100
24          periodSeconds: 15
25  metrics:
26    - type: Pods
27    pods:
```

```
28     metric:
29       name: http_requests_qps
30     target:
31       type: AverageValue
32       averageValue: 50000m
```

这里的配置有三个需要注意的地方。

1. 最小最大的副本数分别设置 1、10。
2. 为了测试效果的时效性，设置扩缩容的行为 behavior。
3. 指定指标 http_requests_qps、类型 Pods 以及目标值 50000m（它表示平均每个 pod 的 RPS 50）。比如以 300 的 RPS 访问，副本数就是 $300/50=6$ 。

测试工具我们选用 `vegeta`，因为它可以指定 RPS。

 复制代码

```
1 # 240
2 echo "GET http://192.168.1.92:31617" | vegeta attack -duration 60s -connections 1
3 # 120
4 echo "GET http://192.168.1.92:31617" | vegeta attack -duration 60s -connections 1
5 # 40
6 echo "GET http://192.168.1.92:31617" | vegeta attack -duration 60s -connections 1
```

我们 describe 下 deployment，就可以看到这样的日志，horizontal-pod-autoscaler New size: 2; reason: pods metric http_requests_qps above target。

```
Metrics: ( current / target )
 resource memory on pods (as a percentage of request): 2% (5476352) / 500%
 resource cpu on pods (as a percentage of request): 2% (1m) / 500%
Min replicas: 1
Max replicas: 3
Deployment pods: 1 current / 1 desired
Conditions:
  Type            Status  Reason                        Message
  ----            -
AbleToScale      True    ReadyForNewScale             recommended size matches current size
ScalingActive    True    ValidMetricFound             the HPA was able to successfully calculate a replica count from memory resource utilization (percentage of request)
ScalingLimited   False   DesiredWithinRange           the desired count is within the acceptable range
Events:
  <none>
```

 极客时间

如果日志如上图所示，就说明配置成功了。

HPA 的不足

回顾刚才的过程，我们发现，基于 Prometheus-adapter 的 HPA 整体功能还是比较完全的，但也有几个不足之处。

第一，反应慢半拍。HPA 是根据历史数据来扩缩容的，所以可能来不及应对突然增加的负载，导致服务中断。

第二，服务抖动问题。HPA 频繁扩缩容 Pod 数量，可能会导致服务响应时间变长，影响用户体验。我在生产环境中遇到过用户明确不要启用自动扩容的情况，因为他们的应用不能很好的兼容水平扩展。

第三，配置复杂。HPA 的基础配置比较简单，但是要扩展它的能力就需要一定的 Kubernetes 知识，小白用户可能搞不定。比如 Prometheus-Adapter 依赖 Prometheus，这要求我们先要维护一套 Prometheus，同时配置上需要指定要转换的指标、转换规则等。如果配置不正确，可能会导致 HPA 无法正常工作。

第四，应用场景有限。HPA 只适用于可以根据资源指标进行扩缩容的应用。不能根据事件来进行扩缩容。

VPA

前面我们学习的 HPA 用于水平扩展，那么垂直扩展 VPA 又适合用在什么样的场景，它又具备什么样的优势呢？

我们以 Prometheus 为例来讨论一下。刚开始集群应用的 Pod 数量比较少，一个 4 CPU、4GB 内存的 Prometheus Pod 就够用了。不过，随着集群规模的扩大，Pod 数量越来越多，Prometheus 需要处理大量涌入的指标，那么性能就会成为瓶颈。

但是 Prometheus 不支持水平扩展，这时，VPA 就能派上用场了。它可以根据单个 Pod 的负载情况，自动扩充其 CPU 和内存资源，有效解决性能问题，同样类似的场景在 DevOps 常用的 Jenkins 上也存在。

所以 Pod 的纵向自动扩缩 VPA 具有以下优势。

不需要再预估 Pod 初始启动的时候要分多少的 CPU 和内存，一切交给 VPA 去处理。

VPA 会根据节点剩余的资源量来确定分配多少 CPU 和内存给 Pod，最大化地利用节点的可分配资源。

VPA 可以自动调整 Pod 的 CPU 和内存请求，无需手工干预。。

但是我们在实际的生产中用上 VPA 的机会并不大，为什么呢？因为垂直扩缩容面对的问题比水平扩缩容要复杂很多。

首先，VPA 不像 HPA，HPA 不会更改正在运行的 Pod，而 VPA 会更新正在运行的 Pod 资源配置，这会导致 Pod 的重建和重启。而且 Pod 有可能被调度到其他的节点上，这有可能会中断应用程序，这点可以说是 VPA 落地的最大的阻碍。

其次，预测资源需求难度比较高。有的时候，应用负载在不断地变化，Pod 需要多少资源可能不是线性增长的。这种情况下 VPA 要想合理地扩缩容，就需要准确预测 Pod 的资源需求，这件事并不容易。

最后，有些应用程序并不支持 VPA 方式。例如基于 JVM 的工作负载是无法使用 VPA 的，因为对此类工作负载的实际内存用量并没有暴露出来。

所以社区对 VPA 的推进比较慢，2017 年社区开始讨论 VPA 的方案，但是一直到 kubernetes 1.25 这个版本，VPA 才演进到 1.0 的版本。

总结

云资源浪费是一个普遍存在的问题，无论是在公有云还是私有云环境中都可能发生。今天，我们分析了造成云资源浪费的原因，主要是资源利用率低和资源浪费两种情况。

我们解决云资源浪费的关键手段是引入动态扩缩容。动态扩缩容是指根据应用的负载情况自动调整资源配置，以满足应用的需求。在 Kubernetes 中常用的两种动态扩缩容工具是水平扩缩容 HPA 和垂直扩缩容 VPA。

这一讲里，我也结合例子，为你讲解了如何使用 Prometheus-Adapter 来扩展 Kubernetes HPA 的能力，支持更多的扩缩容指标。建议你参考课程里的讲解，课后自己动手练习实践一下。

但是 HPA 和 VPA 并不是完美的解决方案，都存在一些局限。例如原生的 HPA 只支持基于 CPU 和内存的指标进行水平扩容，VPA 可能会造成应用的中断等。我们需要根据实际需求选择合适的工具，并进行合理配置。

思考题

今天的课程里，我提到了 HPA 的一些不足，比如在真正流量突发的时候，有可能会遇到反应慢半拍的情况，无法及时跟上扩展，导致应用服务中断。你有哪些思路来解决这个问题呢？

欢迎在评论区与我讨论。如果这一讲对你有启发，也欢迎分享给身边更多朋友。

AI智能总结

1. 云资源浪费的原因包括过度或错误配置、资源预留问题以及不必要的服务，这导致资源闲置浪费。
2. 在云原生体系中，动态扩缩容是解决资源浪费问题的关键，包括应用本身的水平扩展与纵向扩展以及 Kubernetes 集群本身容量的动态扩缩容。
3. Kubernetes HPA (HorizontalPodAutoscaler) 可以根据 CPU 利用率、内存利用率和其他自定义指标自动扩缩容 Pod 数量，以满足应用程序的负载需求。
4. HPA支持从其他的API中获取指标来进行扩容，通过Prometheus-Adapter可以将 Prometheus 监控的应用程序的指标暴露给 Kubernetes HPA。
5. Prometheus-Adapter是一款用于将 Prometheus 指标转换为 Kubernetes 自定义指标的工具，可以实现自动扩缩容 Pod 以满足应用程序的负载需求。
6. 通过配置HPA，可以指定指标类型和目标值，实现根据特定指标动态扩缩容 Pod 数量。
7. 为了测试效果的实效性，可以设置扩缩容的行为 behavior，以及指定指标类型和目标值来验证扩缩容效果。
8. Kubernetes集群支持的第三方API可以提供更多的指标来进行扩缩容，从而更精确地满足应用程序的负载需求。
9. 通过动态扩缩容，可以提高资源利用率，并降低成本，是解决云资源浪费问题的有效途径。
10. 通过动态扩缩容，可以实现在流量高峰时期动态扩容，流量进入低峰时释放闲置资源，提高资源使用率。

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。



cfanbo

2024-04-19 来自江苏

事件驱动动态扩缩容方案用knative方案吗？

作者回复: Knative里有两种方式

第一种叫serving，是根据访问请求来做的动态扩缩容

第二种叫eventing，是事件驱动模型的serverless

共 2 条评论 >



夜空中最亮的星

2024-04-19 来自北京

根据业务情况和历史数据 定时提前扩容。

VPA 有什么示例吗？

作者回复: VPA基于历史数据，推荐资源配置，因此在初期可能不太准确。它没有定时提前的能力。



Starduster

2024-07-01 来自日本

老师好，请问VPA做不到对pod资源限制的热更新吗？假设host资源足够，是不是只需要更改k8s记录的资源限制字段就行了，一定要重启pod才能生效吗？



斯蒂芬.赵

2024-09-04 来自山东

首先，VPA 不像 HPA，HPA 不会更改正在运行的 Pod，而 VPA 会更新正在运行的 Pod 资源配置，这会导致 Pod 的重建和重启。而且 Pod 有可能被调度到其他的节点上，这有可能会中断应用程序，这点可以说是 VPA 落地的最大的阻碍

这句话有毛病吧，VPA 不会中断正在运行的 pod。VPA 的策略是通过 gradual rolling updates 实现平滑地应用升级，避免影响 pod 的运行。

